- Chapter IV: implementing authentication
    - JWTs (Json Web Tokens)
        - The problem
        - A deeper dive
        - The solution
        - Trying to sneak past the signature verification
        - How JWTs fit into OAuth2
    - Bringing everything into shape
        - Modifying the microservices
        - What changed
        - Modifying requests between microservices
        - Modifying the aggregator service
        - Running everything
        - Trying the Authorization Flow with Postman
    - Injecting custom logic
        - Our use case
        - Cross cutting concerns with security rules

Chapter IV: implementing authentication

The reference for this chapter is oauth2-spring-boot/keycloak-cross-cutting, you can use the collection named `oauth2-spring-boot-auth.json` if you want to follow along with Postman.

Note: this chapter requires basic knowledge about the following topics:

- https://en.wikipedia.org/wiki/RSA_(cryptosystem)
- https://en.wikipedia.org/wiki/HMAC

Now that we have our base system working correctly, it's time to plug the OAuth2 layer in and start authenticating requests.

JWTs (Json Web Tokens)

Remember what we said about JWTs? JWTs are cryptographically signed strings that encode information in JSON format. That's a nice definition that sums everything a JWT is in very few words, but what exactly does it mean?
JWTs enable developers to share JSON objects (any JSON object, really) between applications in a way that's *non repudiable* and *authenticated*, meaning that only clients that have the **private key** can **emit** tokens, whereas clients that have the **public key** can *verify* the authenticity of such tokens (this is basic RSA).
What's really groundbreaking about JWTs is that you can encode whatever information you want inside them, making them completely *stateless*.
To better understand the power of JWTs, let's imagine you're designing a login system for your web application that implements the following authentication flow:

- the user inserts their username and password;
- if the credentials are correct a session is created. You represent the session using a random string that you save inside the database and you link it to the authenticated user using the following table:

```
sessions table:
    +---------+------------+
    | user_id | session_id |
    +---------+------------+
    |      1  | abcde1234  |
    +---------+------------+
```

- the session token is stored inside the user's cookies: this way, every time the users issues a request to your server, the session token is sent to the server;
- once the server receives a request, it reads the cookies, gets the session token and uses the session token to perform a lookup query to retrieve the requesting user's information. If we suppose that users are stored in the following way:

```
users table:
    +----+-----------+---------+
    | id | name      | surname |
    +----+-----------+---------+
    | 1  | ponglerio | donglis |
    +----+-----------+---------+
```

  a lookup query can be done by joining on the `user_id` like this:

```
    select users.name, users.surname, users.id
        from sessions
        join sessions.user_id = users.id
        where sessions.session_id = 'abcde1234';
```

- this way the server knows which user performs a certain request based on its session id.

What we just described is the "classical" way of representing sessions and associating them with users.
So far so good, right? This is a good and simple approach to the task that, at first glance, works well, we deploy our solution and everyone is happy, for now.

The problem

We start with 1000 users and everything looks good, but what happens when the traffic towards our application scales? What happens with 10x the requests? Our system starts showing signs of bottlenecks: every time we receive a request, we hit the database with a query! Not only that, our users' table has grown, so lookup times are slower and slower for each user that registers to our service! Don't panic yet, JWTs are here to lift this burden from our shoulders; let's see how we can tackle this problem.
We said that every time we want to verify a user's session we need to hit the database with the query we described above, right? That is needed because the session token we send does not encode **any sort of content**, so what if we could find a way to give a meaning to our token?
That's exactly what JWTs are for: they are strings of text that can be decoded in a very efficient manner that store content and contextual information about the request that it is currently being processed by the server.

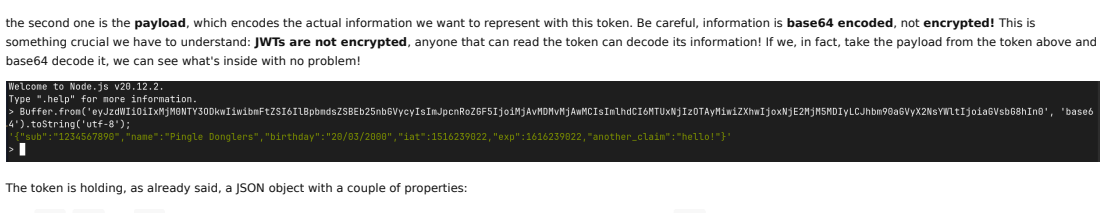
A deeper dive

JWTs look like this:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IlBpbmdsZSBEb25nbGVycyIsImJpcnRoZGF5IjoiMjAvMDMvMjAwMCIsImlhdCI6MTUxNjIzOTAyMiwiYW5vdGhlcl9jb
2KCgESNJT3NGB8QZiCHSxONH9-IUfdXow50
```

They are essentialy composed of three parts, each separated by a dot:

- the first one is the **header**, which encodes (base64) the type of token and the signing algorithm used;
- the second one is the **payload**, which encodes the actual information we want to represent with this token. Be careful, information is **base64 encoded**, not **encrypted!** This is something crucial we have to understand: **JWTs are not encrypted**, anyone that can read the token can decode its information! If we, in fact, take the payload from the token above and base64 decode it, we can see what's inside with no problem!



```
Welcome to Node.js v20.12.2.
Type ".help" for more information.
> Buffer.from('eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IlBpbmdsZSBEb25nbGVycyIsImJpcnRoZGF5IjoiMjAvMDMvMjAwMCIsImlhdCI6MTUxNjIzOTAyMiwiZXhwIjoxNjE2MjM5MDIyLCJhbm90aGVyX2NsYWltIjoiaGVsbG8hIn0', 'base64').toString('utf-8');
'{"sub":"1234567890","name":"Pingle Donglers","birthday":"20/03/2000","iat":1516239022,"exp":1616239022,"another_claim":"hello!"}'
> 
```

  The token is holding, as already said, a JSON object with a couple of properties:

    - `sub`, `iat` and `exp` are standard properties (*registered claims*) that are generally included in every token: `sub` stands for "subject" and it usually represents some sort of identifier (the token consumer's or the token's, it depends), `iat` stands for "issued at", the timestamp the token has been generated at, whereas `exp` stands for "expiry", the timestamp representing the time at which the token will no longer be valid (expired!); tokens without the `exp` claim do not expire. Here's a list of all registered claims;
    - other claims are custom and totally arbitrary pieces of information we can encode in our token (`name`, `birthday`, `another_claim`).
- the third one is the **signature**, the most important piece of the token: this "slice" is generated using a cryptographic algorithm that can be either symmetric (some variation on HMAC, i.e. `HS256`, `HS384` ...) or asymmetric (RSA or elliptic curve based). Whichever algorithm we are using, this bit of the token can only be verified by those who:
    - know the symmetric key that has been used to produce the signed digest, if generated symmetrically;
    - know the public key of the issuer of the token, that signed it with their private key.

Note: the "verification" of a JWT is essentially the process of ensuring the information contained in the payload has not been tampered with, because only the ones that know the cryptographic items to verify and/or sign the token can modify it!
This is by no means an exhaustive guide on JWTs, but that's essentially all there's to it. You can find more details here.

The solution

Ok, great stuff, but how does all of this fit into our first example? Since JWTs **directly** encode information, we can modify our login flow:

- the user inserts their username and password;
- if the credentials are correct a JWT is issued using a symmetric (for simplicity's sake) algorithm containing the following contents:

```
{
    "iat": <whatever>,
    "exp": <whatever> + <delta>,
    "sub": 1,
    "name": "ponglerio",
    "surname": "donglis"
}
```

    - `iat` is set at whatever timestamp the token was generated at;
    - `exp` is set the same date plus a delta (in seconds), meaning that the token will expire `<delta>` seconds after the `iat` timestamp;
    - `sub` is set to `1`, meaning that the token identifies the user with id `1` (that's the id inside our database);
    - `name` and `surname` are complementary pieces of information that can be used for other scopes;
- the JWT is stored inside the user's cookies: this way, every time the users issues a request to your server, the token is sent to the server;
- once the server receives a request, it reads the cookies, gets the JWT and uses the symmetric key to verify it. A successful verification means that the information has not been tampered with and the contents of the token can be consulted safely. Since token signatures are generated using high-entropy algorithms, a small change inside the contents will drastically change the signature the token is holding. More information below.

The server is now not required to lookup the database everytime it receives a request and the session table is gone! This is a much more scalable approach, since it completely eliminates the bottlenecks that were strangling our system's performance.

Trying to sneak past the signature verification

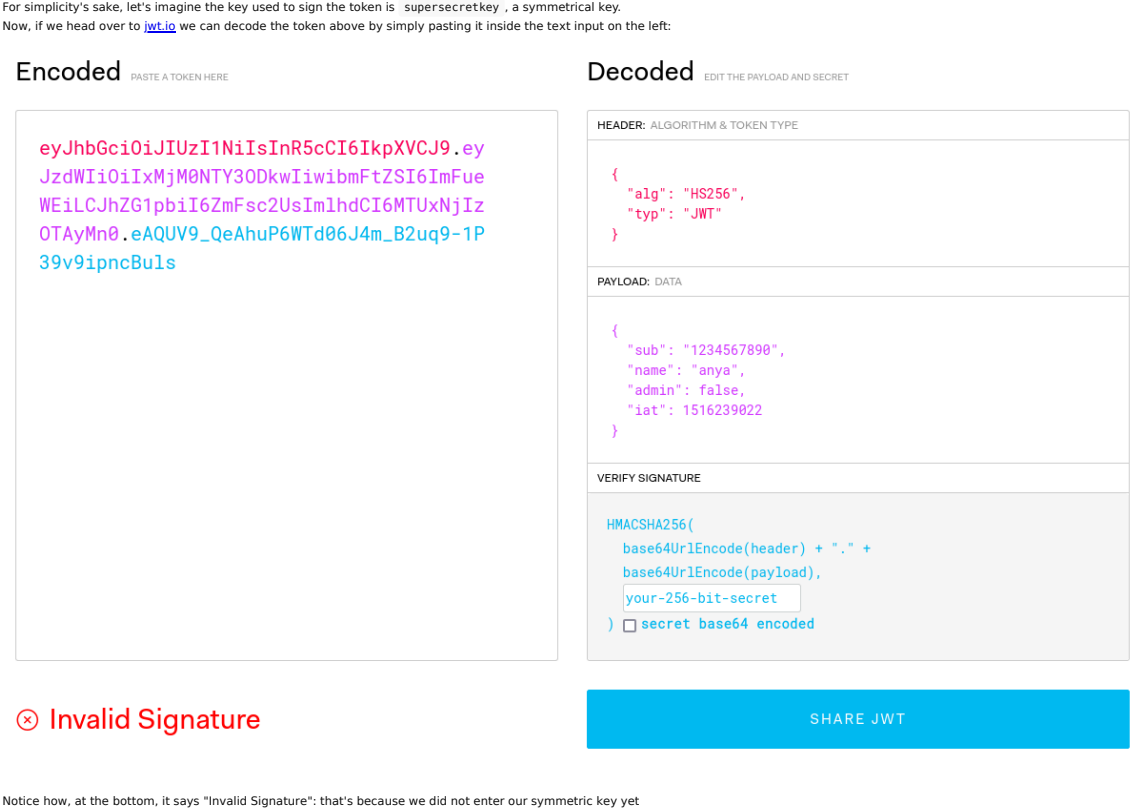
Well, what happens if an attacker changes the information inside the token? Let's see...
Let's imagine the user `anya` logs in and receives the following token:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6ImFueWEiLCJhZG1pbiI6ZmFsc2UsImlhdCI6MTUxNjIzOTAyMn0.eAQUV9_QeAhuP6WTd06J4m_B2uq9-
1P39v9ipncBuls
```

that, if decoded, outputs:

```
{
  "sub": "1234567890",
  "name": "anya",
  "admin": false,
  "iat": 1516239022
}
```

For simplicity's sake, let's imagine the key used to sign the token is `supersecretkey`, a symmetrical key.
Now, if we head over to jwt.io we can decode the token above by simply pasting it inside the text input on the left:



Encoded PASTE A TOKEN HERE
Decoded EDIT THE PAYLOAD AND SECRET
HEADER: ALGORITHM & TOKEN TYPE
PAYLOAD: DATA
VERIFY SIGNATURE
SHARE JWT

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.ey
JzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6ImFue
WEiLCJhZG1pbiI6ZmFsc2UsImlhdCI6MTUxNjIz
OTAyMn0.eAQUV9_QeAhuP6WTd06J4m_B2uq9-1P
39v9ipncBuls
```

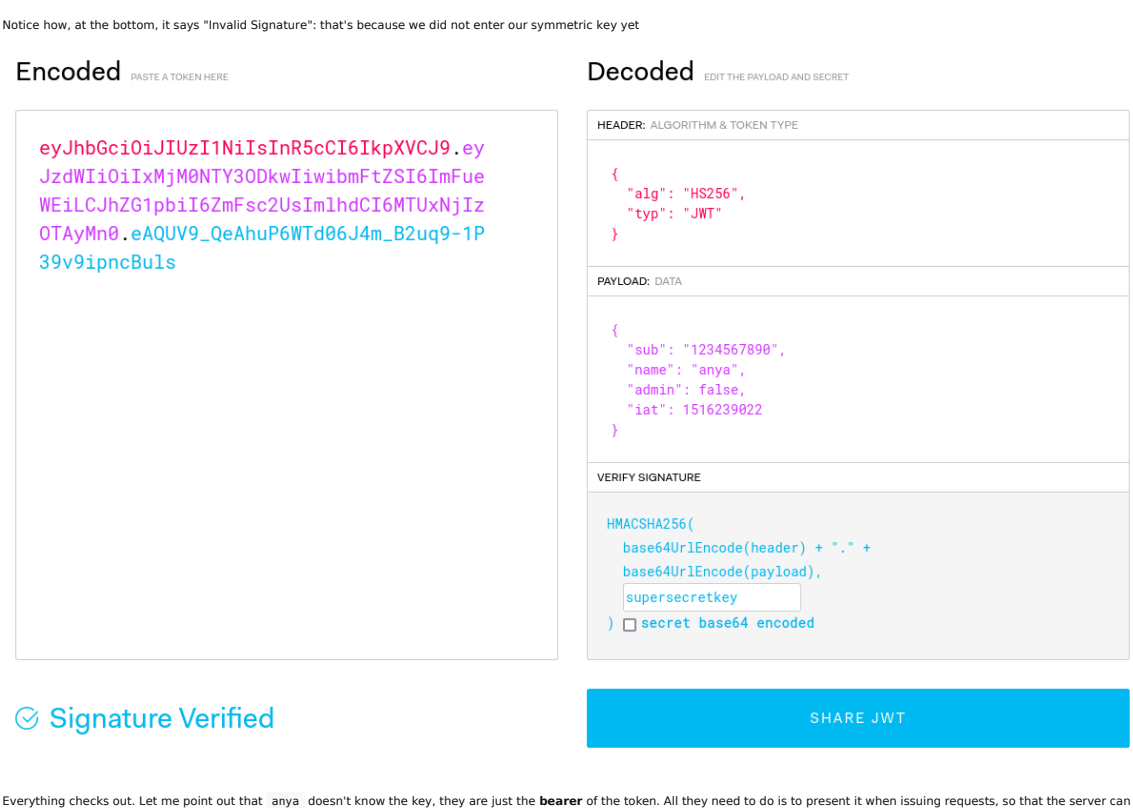
```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

```
{
  "sub": "1234567890",
  "name": "anya",
  "admin": false,
  "iat": 1516239022
}
```

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your-256-bit-secret
) ☐ secret base64 encoded
```

Invalid Signature

Notice how, at the bottom, it says "Invalid Signature": that's because we did not enter our symmetric key yet



Encoded PASTE A TOKEN HERE
Decoded EDIT THE PAYLOAD AND SECRET
HEADER: ALGORITHM & TOKEN TYPE
PAYLOAD: DATA
VERIFY SIGNATURE
SHARE JWT

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.ey
JzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6ImFue
WEiLCJhZG1pbiI6ZmFsc2UsImlhdCI6MTUxNjIz
OTAyMn0.eAQUV9_QeAhuP6WTd06J4m_B2uq9-1P
39v9ipncBuls
```

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

```
{
  "sub": "1234567890",
  "name": "anya",
  "admin": false,
  "iat": 1516239022
}
```

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  supersecretkey
) ☐ secret base64 encoded
```

Signature Verified

Everything checks out. Let me point out that `anya` doesn't know the key, they are just the **bearer** of the token. All they need to do is to present it when issuing requests, so that the server can verify that they logged in *before* making certain calls.

Let's now imagine `anya` gets their PC hacked and outgoing traffic is sniffed and pre-processed before being sent to external parties. Let's imagine, then, a malicious actor trying to elevate `anya` to admin role to gain privileged access to the system.
The attacker forges a new payload...

```
{
  "sub": "1234567890",
  "name": "anya",
  "admin": true, <-- important change!
  "iat": 1516239022
}
```

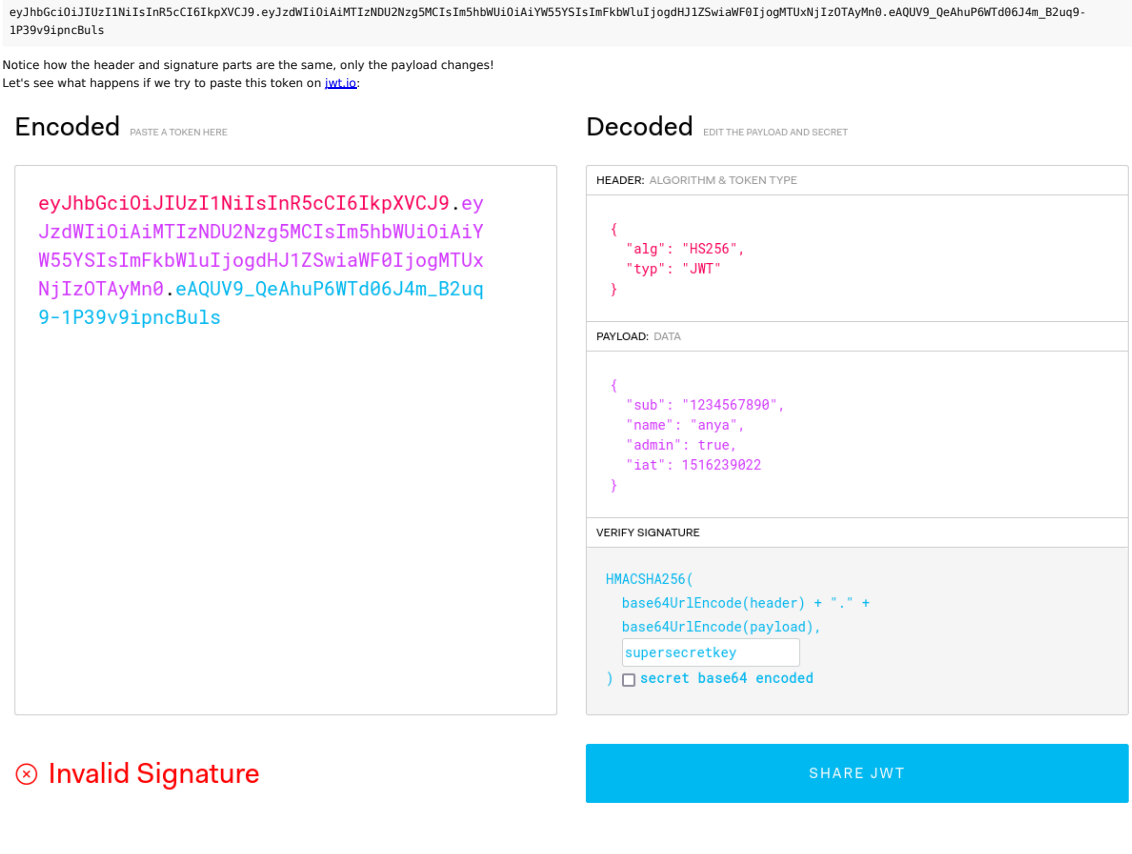
... then they base64 encode it...

```
eyJzdWIiOiAiMTIzNDU2Nzg5MCIsIm5hbWUiOiAiYW55YSIsImFkbWluIjogdHJ1ZSwiaWF0IjogMTUxNjIzOTAyMn0
```

... and finally, they replace the old payload with the malicious one, creating a forged token!

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiAiMTIzNDU2Nzg5MCIsIm5hbWUiOiAiYW55YSIsImFkbWluIjogdHJ1ZSwiaWF0IjogMTUxNjIzOTAyMn0.eAQUV9_QeAhuP6WTd06J4m_B2uq9-
1P39v9ipncBuls
```

Notice how the header and signature parts are the same, only the payload changes!
Let's see what happens if we try to paste this token on jwt.io:



Encoded PASTE A TOKEN HERE
Decoded EDIT THE PAYLOAD AND SECRET
HEADER: ALGORITHM & TOKEN TYPE
PAYLOAD: DATA
VERIFY SIGNATURE
SHARE JWT

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.ey
JzdWIiOiAiMTIzNDU2Nzg5MCIsIm5hbWUiOiAiY
W55YSIsImFkbWluIjogdHJ1ZSwiaWF0IjogMTUx
NjIzOTAyMn0.eAQUV9_QeAhuP6WTd06J4m_B2uq
9-1P39v9ipncBuls
```

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

```
{
  "sub": "1234567890",
  "name": "anya",
  "admin": true,
  "iat": 1516239022
}
```

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  supersecretkey
) ☐ secret base64 encoded
```

Invalid Signature

Ah! The verification process **fails** miserably, and our attacker is not able to fool our system: that's because **the signature did not change when we replaced the payload**! Since the attacker does not know the symmetric key with which signatures are generated, it is mathematically impossible to forge a token that eludes the verification process, because there's no way to guess the correct signature without the key!

How JWTs fit into OAuth2

Remember when we talked about access and refresh tokens? They are asymmetrically-generated JWTs! This means that Keycloak generates and signs such tokens with its private key and publishes its public key to interested parties as a mean to verify issued tokens (we will see how in a bit).
In other words, once our user is authenticated, any microservice that holds Keycloak's public key will be able to verify their token without **ever** contacting Keycloak itself: that's **really** cool and removes **a lot** of coupling from our system!
By leveraging the stateless nature of JWTs we created a completely distributed **authorization layer**!

Bringing everything into shape

If you are reading this paragraph, it is assumed that you have followed the steps described in Chapter I regarding the creation of Keycloak realms and clients.
In this section, the realm we are going to use is called `test-realm`, whereas the confidential clients will be called `spring` for microservices (authorization flow) and `aggregator` for the aggregator service (service account flow). Feel free to change these parameters to your liking.

Remember to change the `KC_HOSTNAME` directive to your IP in the `compose.yml` file! Stuff won't work if you don't do this!!

Modifying the microservices

Because of OAuth2's importance in the field of distributed systems, Spring Boot actually has integrated the standard inside Spring Security. To add the required packages, we just need to install Spring's OAuth2 resource server.

Gradle:

```
implementation("org.springframework.security:spring-security-oauth2-resource-server:6.3.1")
```

Maven:

```
<!-- https://mvnrepository.com/artifact/org.springframework.security/spring-security-oauth2-resource-server -->
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-oauth2-resource-server</artifactId>
    <version>6.3.1</version>
</dependency>
```

It's now time to configure everything to make it work with our Docker environment.
Inside the microservices' `application.yml` file, we must configure our OAuth2 security server to conform to our parameters:

```
# grades microservice
server.port: 8081
server.error.include-message: always
server.error.include-stacktrace: never
students.uri: http://students:8080/

spring:
  ## the important part ##
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://your.local.ip:7778/realms/test-realm # note: this MUST be the "outside facing" URI, not the internal (Docker) one!
          jwk-set-uri: http://keycloak:8080/realms/test-realm/protocol/openid-connect/certs
    ####
  datasource:
    url: jdbc:postgresql://grades_pg:5432/grades
    username: test
    password: test
  jpa:
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
        physical_naming_strategy: org.hibernate.boot.model.naming.CamelCaseToUnderscoresNamingStrategy
---
spring.config.activate.on-profile: docker
server.port: 8080
spring:
  devtools:
    livereload:
      enabled: true
```

```
# students microservice
server.port: 8080
server.error.include-message: always
server.error.include-stacktrace: never

grades.uri: http://grades:8080/

spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://your.local.ip:7778/realms/test-realm
          jwk-set-uri: http://keycloak:8080/realms/test-realm/protocol/openid-connect/certs
  datasource:
    url: jdbc:postgresql://students_pg:5432/students
    username: test
    password: test
  jpa:
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
        physical_naming_strategy: org.hibernate.boot.model.naming.CamelCaseToUnderscoresNamingStrategy
---
spring.config.activate.on-profile: docker
server.port: 8080
spring:
  devtools:
    livereload:
      enabled: true
```

- `issuer-uri` is the URL of our Keycloak instance. This value will be compared with the `iss` field inside the JWT, so be careful to write the exact URI;
- `jwk-set-uri` is essentially the endpoint where Keycloak will publish its public keys in a JSON-friendly (and standard) format and can be found by navigating to the OpenID configuration endpoint. Once started, microservices configured with this URI will automatically download public keys from the identity provider and will be able to verify tokens corresponding to such keys.

With such little configuration we have essentially done everything we need to do to correctly protect our APIs with JWT validation against our Keycloak instance!

What changed

Now that we inserted this package and its configuration, Spring will perform authorization checks on every requests it receives. These checks generally consist of:

- checking the headers (or something else, if configured) for a JWT. Spring will look, by default, inside the `Authorization` header to see if there is any `Bearer` token. This means that every request that reaches our Spring instance must have the authorization header set to the value `Authorization: Bearer <token>`. If, for example, our token was `aaabbbccc` our authorization header would look like this: `Authorization: Bearer aaabbbccc`. Spring would then extract the token and run the remaining checks. If no token is present, the request is automatically dropped;
- checking the validity of the token and other claims, such as the issuer;
- applying custom authorization logic (we will do it later). Here we can instruct Spring to perform other checks and/or search JWTs in other parts of the request (such as cookies).

Modifying requests between microservices

Now that we enabled authorization on all microservices, we also have to authenticate requests made *between* microservices.
To add the authorization logic we talked in the previous paragraph, we must support the insertion of an authorization header in our request.
The question now is, how can a microservice obtain the signed JWT that can enable it to perform authenticated requests to other microservices? Simple: it does not! Since every request that arrives to our application must already be authenticated through some form of JWT, microservices can use that JWT to authenticate other request they have to make!
Let's imagine a scenario where there is an authenticated user that issues a request to microservice A and that microservice must perform another request to microservice B, forming a chain of two requests.
The request between the user and A is authenticated, since the user presents the token to the microservice, so that works. Now, A must issue a request to B: to do so, it extracts the user's token from the previous request and uses it to perform the new one. This method not only is simple, it also enables our system to track the request of the user throughout the whole system, making it easy to log and trace users.
An additional benefit of this method is that permissions that have been given to users are enforced in every request of the chain! Let's imagine our user has the following token:

```
{
  "name": "anya",
  "admin": false,
  "can_access_a": true,
  "can_access_b": false
}
```

If the user has certain permissions, such as in this case, these are enforced throughout the whole chain of requests. In this case, if microservice B checked the token for the truthiness of `can_access_b`, our user wouldn't be able to access it, because their token gets passed along each node of the request chain;

Now that we introduced this concept, let's modify our microservices, particularly the grades microservice.
Upon inserting a grade, in fact, the grade microservice issues a request to the students microservice to check if a student exists before inserting the grade inside its database:

```
@PostMapping(value = "/{studentId}")
    public SuccessResponse<String> addGradeToStudentId(
            @PathVariable String studentId,
            @RequestBody AddGradeRequest body
    ) {
        studentsService.getStudent(Integer.parseInt(studentId));
        grades.insertGrade(body.getGrade(), Integer.parseInt(studentId));
        return new SuccessResponse<String>("grade added");
    }
```

Where, `getStudent` is:

```
public Student getStudent(int studentId) {

    return this.http
            .get()
            .uri("/{id}", studentId)
            .retrieve()
            .onStatus(HttpStatusCode::is4xxClientError, (req, res) -> {
                throw new StudentNotFoundError(studentId);
            })
            .body(Student.class);
}
```

As we can see, there is no track of headers and authorization. We can add our required functionality by making very few modifications.
We can retrieve the token from the request in this way:

```
@PostMapping(value = "/{studentId}")
public SuccessResponse<String> addGradeToStudentId(
        @PathVariable String studentId,
        @RequestBody AddGradeRequest body,
        // we extract the authorization header here!
        @RequestHeader(HttpHeaders.AUTHORIZATION) String authorizationHeader
) {
    // we are sure the header exists and is valid, because the jwt checks already passed!
    studentsService.getStudent(Integer.parseInt(studentId), authorizationHeader);
    grades.insertGrade(body.getGrade(), Integer.parseInt(studentId));
    return new SuccessResponse<String>("grade added");
}
```

and `getStudent` becomes:

```
public Student getStudent(int studentId, String bearer) {
    return this.http
            .get()
            .uri("/{id}", studentId)
            .header("Authorization", bearer)
            .retrieve()
            .onStatus(HttpStatusCode::is4xxClientError, (req, res) -> {
                throw new StudentNotFoundError(studentId);
            })
            .body(Student.class);
}
```

That was pretty easy!

Modifying the aggregator service

Until now, the aggregator has been making unauthenticated requests: we need to fix this, because otherwise the service would not be able to reach our microservices.
Let's add the authentication/authorization logic:

```

...
private <C> C fetchData(String url, Map<String, String> headers, FormBody body, Class<C> classObj) throws IOException, NotOkException {
    Request req = buildRequest(url, headers, body);
    Response res = client.newCall(req).execute();
    ResponseBody resBody = res.body();
    if(res.code() != 200){
        throw new NotOkException();
    }
    assert(resBody != null);
    Gson gson = new Gson();
    String strBody = resBody.string();
    return gson.fromJson(strBody, classObj);
}

private void performAuthentication() throws NotOkException, IOException {
    // short lived credentials for the authenticator!
    String clientSecret = System.getenv("CLIENT_SECRET");
    String clientId = System.getenv("CLIENT_ID");
    String authUri = System.getenv("AUTH_URI");

    FormBody body = new FormBody.Builder()
            .add("client_id", clientId)
            .add("client_secret", clientSecret)
            .add("grant_type", "client_credentials")
            .build();
    AuthResponse auth = fetchData(authUri, null, body, AuthResponse.class);
    this.bearerToken = auth.accessToken;
}
...
@Override
public void run() {
    String gradesUri = System.getenv("GRADES_URI");
    if(gradesUri == null){
        gradesUri = "http://localhost:8081";
    }

    String studentsUri = System.getenv("STUDENTS_URI");
    if(studentsUri == null){
        studentsUri = "http://localhost:8080";
    }

    try{
        StudentsResponse allStudents = this.fetchData(studentsUri, Map.of("Authorization", "Bearer " + this.bearerToken), null, StudentsResponse.class);
        GradesResponse allGrades = this.fetchData(gradesUri, Map.of("Authorization", "Bearer " + this.bearerToken), null, GradesResponse.class);

        final List<ReportCard> cards = Arrays.stream(allStudents.result).map(student -> {
            List<Grade> studentGrades = Arrays.stream(allGrades.result).filter(grade -> grade.studentId == student.id).toList();
            return new ReportCard(student.name + " " + student.surname, studentGrades);
        }).toList();

        cards.forEach(card -> {
            System.out.println(card.toString());
        });

    }catch(MalformedURLException e){
        System.out.println("cannot perform task because the url is malformed");
    }catch(IOException e){
        System.out.println("cannot perform http request: ");
        e.printStackTrace();
    }catch(NotOkException e){
        System.out.println("unauthorized!");
        try {
            performAuthentication();
        } catch (Exception ex) {
            System.out.println("couldn't authenticate, is there something wrong with the idp?");
        }
    }
}
...
```

We can see that we added a fair bit of code:

- we modified the `fetchData()` function from the base branch and we added the response code check: this way, if the response code is not 200, an exception will be thrown;
- we added the `performAuthentication()` method, that essentially performs the **service account flow**;
- the `run()` method is essentially the same, we added the catch-block in case the request we make uses invalid credentials.

Fantastic, our aggregator is now ready to make authenticated requests! Let's now see how to start everything.

Running everything

Let's fire up our containers and see what's up. We can see in our logs that the `aggregator` service is erroring, saying:

```
aggregator-1   |{Authorization=Bearer }
aggregator-1   | unauthorized!
```

This proves that our API needs some sort of authorization before being used! Nice!
To fix this issue you must create a **service account** confidential client and set the environmental values in the `compose.yml` file as such:

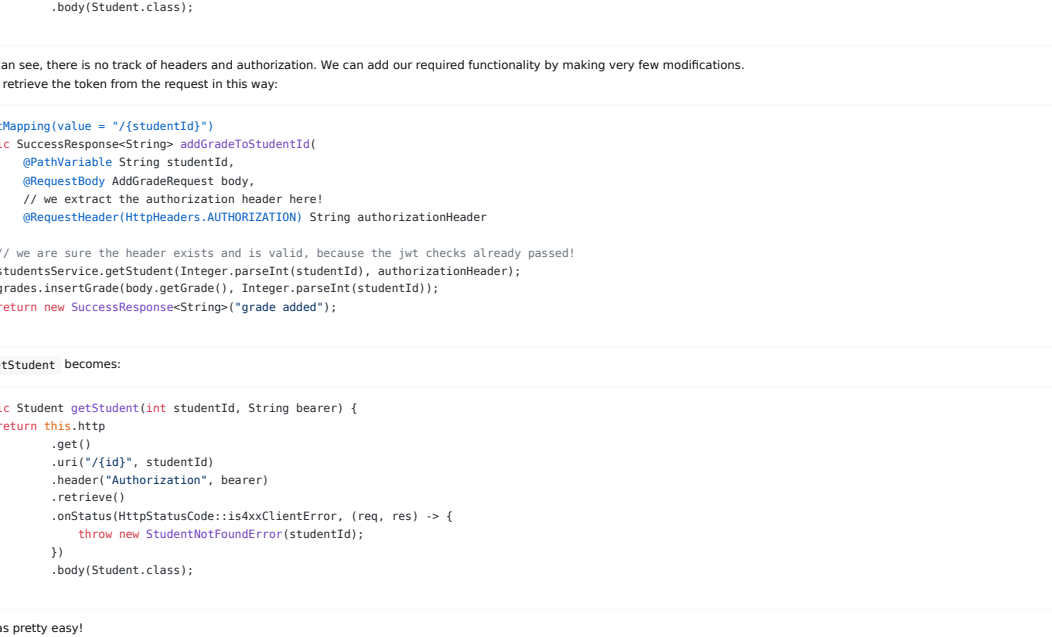
```
...
aggregator:
    depends_on:
      - students
      - grades
      - keycloak
    networks:
      - microservices-net
    build:
      context: microservices/aggregator
      dockerfile: Dockerfile
    mem_limit: 512m
    environment:
      - STUDENTS_URI=http://students:8080
      - GRADES_URI=http://grades:8080
      - AUTH_URI=http://keycloak:8080/realms/master/protocol/openid-connect/token
      - CLIENT_SECRET=the-secret-you-got-from-keycloak
      - CLIENT_ID=aggregator
...
```

If you did all these steps correctly, your aggregator should now be logging the token it got from the idp and all the report cards from the microservices.

Trying the Authorization Flow with Postman

Now that we setup everything, we can try to simulate a user trying to use our service with Postman.

1. let's fetch our credentials from Keycloak as we described in Chapter I (supposing you correctly set up the realm);
    - Navigate to `http://KEYCLOAK_IP:KEYCLOAK_PORT/realms/test-realm/protocol/openid-connect/auth?response_type=code&client_id=spring`
    - Authenticate and grab the `code` from the url;
    - perform a `POST` request to `http://KEYCLOAK_IP:KEYCLOAK_PORT/realms/test-realm/protocol/openid-connect/token`. The request should look like this with, of course, your credentials inserted:



POST {{idp}}/realms/test-realm/protocol/openid-connect/token

○ none ○ form-data ● x-www-form-urlencoded ○ raw ○ binary ○ GraphQL

	Key	Value	Description
☑	client_id	example-spring	
☑	client_secret	CaR0PujNYnFLomBSrncZsiwlDllltftD	
☑	grant_type	authorization_code	
☑	code	de776ca7-81e3-40c1-b905-23000b03d019.67ca705f...	
	Key	Value	Description

    - you should now have your `access_token`.
2. Using the access token you got from the previous step, you can try to make a request to the API. You should also try doing the same request **without** the token - you will see a 401 error, meaning that you can't make unauthorized requests to the API anymore.

Injecting custom logic

You can find the reference code regarding this paragraph here.

We can modify how Spring performs authorization checks by creating custom beans inside our application code.
Right now, Spring Web Security will ensure the validity of the token, meaning that the cryptographical integrity and its expiration date will be checked by default.
To extend this behaviour, we can inject our custom logic using the `@Configuration` annotation.
Let's create a class, called `OAuth2Configuration`:

```
@Configuration
@EnableWebSecurity // this decorator allows us to configure spring web security
public class OAuth2Configuration {
  ...
}
```

To implement security rules, Spring Web Security uses what's called a *security chain*, a mechanism similar to the one that `iptables` uses: to process a request, Spring executes a list of rules that compose the security chain and, if one of these throws an error, the request will be rejected.
We can add a rule - a link, if you will - to our security chain by using the `SecurityFilterChain` bean:

```
@Configuration
@EnableWebSecurity
public class OAuth2Configuration {

    @Value("${spring.security.oauth2.resourceserver.jwt.issuer-uri}")
    private String issuerUri;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
      // HttpSecurity exposes the configuration API
      ...
    }
  ...
}
```

Inside the `filterChain` method, we will override the "default" chain by adding our custom rules:

```
@Configuration
@EnableWebSecurity
public class OAuth2Configuration {
    ...
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        // default chain starts here
        http
                .authorizeHttpRequests(authorize -> authorize
                .anyRequest().authenticated()
        )
        // default chain ends here
        // custom rules start here
                .something(myRules());
        return http.build();
    }
    ...
}
```

Our use case

Since we can create custom rules, let's imagine we want to limit access to users that have their email verified. If we take a look to the `access_token` we receive with each request, we notice that Keycloak embeds this information inside the JWT:

```
{
  "exp": 1722963232,
  "iat": 1722962932,
  ... more stuff
  "email_verified": false, <-- here!
  "client_id": "some_client_id"
  ... more stuff
}
```

Inside our Spring configuration we can inject logic that reads our JWT, verifies the claim and decides what to do with the request. To do so, we create a `EmailValidator` class that we will later use inside our `OAuth2Configuration` bean:

```
public class EmailVerifiedValidator implements OAuth2TokenValidator<Jwt> {
      OAuth2Error error = new OAuth2Error("email_not_verified", "You must verify your email to continue!", null);

      @Override
      public OAuth2TokenValidatorResult validate(Jwt token) {
          boolean isEmailVerified = token.getClaimAsBoolean("email_verified");
          if(!isEmailVerified){
              return OAuth2TokenValidatorResult.failure(error);
          }
          return OAuth2TokenValidatorResult.success();
      }
}
```

This class extends the `OAuth2TokenValidator` which is one of the many generic classes that we can use to alter the rule flow in Spring's security chain. Upon receiving a request, in fact, each security chain class' `validate` method will be called: if any of them fails, the security chain will stop processing rules and the request will be rejected.
In this example, the `validate` method receives a `Jwt` instance (because it is a `OAuth2TokenValidator<Jwt>`) that we can use to implement our custom logic: we get the `email_verified` claim, we check for its truthiness and, if the claim is `null` or `false`, we reject the request.
We almost forget about the `aggregator` client! Service accounts do not have email associated to them, at least not in our case, so we must skip the check in case we receive a request from the aggregator.
To verify the client's name we can check the `client_id` claim:

```
public OAuth2TokenValidatorResult validate(Jwt token) {
    String clientId = token.getClaimAsString("client_id");
    if(clientId.equals("aggregator")){
        // we skip the check here
        return OAuth2TokenValidatorResult.success();
    }
    boolean isEmailVerified = token.getClaimAsBoolean("email_verified");
    if(!isEmailVerified){
        return OAuth2TokenValidatorResult.failure(error);
    }
    return OAuth2TokenValidatorResult.success();
}
```

To support these kind of checks, Spring uses something called `JwtDecoder` security rules. To add a `JwtDecoder` we need to create a bean that embeds our custom logic, we can do that by adding a bean to our `OAuth2Configuration` class:

```
@Bean
JwtDecoder rolesDecoder(){
    // this creates a JwtDecoder from the configured issuer.
    // you can read more here: https://docs.spring.io/spring-security/site/docs/current/api/org/springframework/security/oauth2/jwt/JwtDecoders.html
    NimbusJwtDecoder jwtDecoder = (NimbusJwtDecoder) JwtDecoders.fromIssuerLocation(issuerUri);
    // we create our EmailValidator
    OAuth2TokenValidator<Jwt> withEmailCheck = new DelegatingOAuth2TokenValidator<>(new EmailValidator());
    // we add it to the jwt decoder
    jwtDecoder.setJwtValidator(withEmailCheck);
    return jwtDecoder;
}
```

To add the jwt decoder to the security chain, we must use the `oauth2ResourceServer` subchain (the part of the security chain dedicated to jwt-related checks):

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
  http
      .authorizeHttpRequests(authorize -> authorize
      .anyRequest().authenticated()
  )
  // we add this subchain checks
  .oauth2ResourceServer(oauth2 ->
          oauth2.jwt(jwt ->
                  jwt.decoder(rolesDecoder())) // here's our decoder
  );
  return http.build();
}
```

And there we have it! Custom checks and rules inside the JWT chain, unleashing the true power of the Spring Security library!

Cross cutting concerns with security rules

At this stage of the project, we successfully created a flock of microservices that implement custom security rules for authentication.
We already noticed, though, that we had to copy our security rules on every microservice (in this case we only have two, but imagine having more than 10!), a process that is not only boring and cumbersome, but also very error prone!
There are tools such as Lombok that help us reducing boilerplate and keeping our code more maintainable, but such tools are language-dependant: what if we had microservices written in different languages? How could we implement such security rules in a way that is uniform and language-agnostic across microservices? Well, if you are curious, you should definitely check Chapter V, the last one, where we will create a single layer in our infrastructure that will be responsible of handling the authorization and authentication of our requests.

Next chapter: Chapter V: for the fearless — eliminating cross cutting concerns from the authorization layer with NGINX
Previous chapter: Chapter III: base project